

CLASH OF CROWNS

Final Connected Engine + Gameplay + Progression Implementation Plan

Rust + TypeScript Only | No JavaScript Engine Logic | Playability Acceptance Guide

Document Field	Final Value
Status requested by owner	Connected final plan after all client feedback and clarifications
Primary goal	Make the engine, rules, levels, academy, stats, UI and deployment ready according to requirement
Backend engine language	Rust
Frontend language	TypeScript only
JavaScript engine logic	Not allowed in production
Current release decision	Internal QA / implementation pending after this plan; public release only after all tests pass
Non-negotiable source rule	Use uploaded instructions as source of truth. Do not add Core or Promotion Trial.

Read This First: What This PDF Guarantees

- This PDF combines the earlier bug reports, new WhatsApp/file instructions, and the final clarification about level engines, loss rules, cup rules, ratings and architecture.
- It gives an exact implementation plan and copy-paste prompts for the coding agent.
- No document alone can guarantee the game will be playable without actually implementing and testing the code. The guarantee condition is: if the agent implements this plan exactly and all automated plus manual acceptance tests pass, then the engine/gameplay will match the stated requirement.
- This document intentionally freezes the architecture: Rust engine, TypeScript UI/adapters, no JavaScript chess engine logic in production.

Source-of-Truth Files and How They Must Be Used

Source	Use in final project	Important interpretation
AMchess.txt / AMchess-1.txt	Level order, rating ranges, number of characters, win/loss rules.	Final level order is Beginner, Learner, Intermediate, Hard, Master, Grandmaster. Remove Core and Promotion Trial.
AI_rules.txt	Rules reference only.	Convert the rule behavior into Rust using a Rust chess library. Do not paste HTML/JavaScript into production.
cup systems + names.txt	Bot names and cup designs.	Use only the last cup method: Round Robin / Point Table. Do not use Knockout or Ladder.
NNUE_rust.txt	NNUE profile, depths, error/noise pattern, HalfKP-style guidance.	Use as Rust-side engine profile reference; adapt to current tensor loader safely.
nega+alphabeta+hce.txt	HCE + Negamax + Alpha-Beta reference.	Convert concepts into Rust and TypeScript types only. Do not add JS engine runtime.
be+le_locksystem+zeroglobalvariable+HCE+nega+ABprune.txt	Beginner/Learner lock system and pure-state design reference.	Use the concept but correct Learner loss rule to 3 consecutive losses as final owner clarification.
Client screenshots / WhatsApp feedback	Bug list and user experience blockers.	Must be treated as QA blockers before public release.

Global Non-Negotiable Rules

- Frontend must be TypeScript only. No JavaScript chess engine, no HTML/JS rule file running in production.
- Backend/engine must be Rust. Use shakmaty or an equivalent Rust chess rules library for legal move validation and FEN handling.

- TypeScript is allowed for UI, routing, settings, API adapters, localization, state display, and client-side persistence UI. It is not allowed to calculate real engine moves.
- Rust is responsible for legal move generation, check, checkmate, stalemate, castling, en passant, promotion, repetition, 50-move rule validation, engine selection, evaluation and bot move response.
- Do not re-add Core or Promotion Trial. They are irrelevant to the final requirement.
- Do not change design randomly. Fix behavior while keeping premium dark/gold identity unless a bug requires a small UI correction.
- Stockfish is not gameplay primary. Stockfish can remain benchmark/debug only if already present.
- All new features must include tests and manual verification notes.

Final Level Order and Engine Routing

Order	Level	Engine model	PST rule	Neural/NNUE	Search	Error/noise rule	Loss rule
1	Beginner	SCE / static evaluation	PST only for Pawn, Bishop, Knight	No	Simple shallow search / legal random assisted	Weak variety allowed; avoid stupid loops	Loss = stay same level
2	Learner	SCE / static evaluation	PST for all pieces	No	Shallow Negamax/Alpha-Beta allowed	Controlled weak randomness	3 consecutive losses = down 1 level
3	Intermediate	NNUE / neural evaluation	NNUE features; PST not primary	Yes	Negamax + Alpha-Beta	Error around 0.402 then decreasing	2 consecutive losses = down 1 level
4	Hard	NNUE / neural evaluation	NNUE features; PST not primary	Yes	Negamax + Alpha-Beta	Error around 0.154 down toward near zero	1 loss = down 1 level
5	Master	NNUE / neural evaluation	NNUE features	Yes	Negamax + Alpha-Beta	Slight error only	Cup rules apply
6	Grandmaster	NNUE / neural evaluation	NNUE features	Yes	Negamax + Alpha-Beta	Zero error, deterministic	Final elite tier rules

Final Rating / ELO Structure

This table removes the old uneven values such as 50, 200, 150, 220. Ratings must increase cleanly and should match the level ranges from the instructions.

Level	Bot count / format	Rating range	Recommended ratings	Notes
Beginner	5 bots	300-700	300, 400, 500, 600, 700	Use SCE + PST on Pawn/Bishop/Knight. First bot unlocked by default.
Learner	5 bots	700-1000	760, 820, 880, 940, 1000	Use SCE + all-piece PST. 3 consecutive losses down one level.
Intermediate	8 bots	1100-1500	1100, 1160, 1220, 1280, 1340, 1400, 1450, 1500	NNUE + search. Error starts near 0.402 and lowers.
Hard	8 bots	1500-2000	1550, 1620, 1690, 1760, 1830, 1900, 1950, 2000	NNUE + search. 1 loss sends back one level.
Master	3 cups	2000-2500	Bronze Cup 2100, Silver Cup 2300, Gold Cup 2500	Each cup has 3 AI bots + 1 real player in Round Robin.
Grandmaster	Final elite bots	2500+	2600, 2700, 2800, 2900, 3000 if 5 bosses are used	Zero error. Exact bot names/count need final client source if different.

Recommended Bot Names and Profiles

Level	Bot names from connected source plan	Engine depth / behavior
Beginner	Woodpecker, Gambit-Bot, Pawn-Pioneer, Castler, En-Passant-X	Depth 1-3; weak but legal; anti-loop; SCE + PST on Pawn/Bishop/Knight.
Learner	Squire Tactics, Bishop Shadow, Flank-Raider, Fianchetto, Paladin-AI	Depth 2-4; all-piece PST; 3-loss downlevel.
Intermediate	The Rook-Project, Zugzwang, Morphy-Core, Caissa Eye, Hyper-Modern, Centaur-V1, Intermezzo, Endgame-Architect	Depth 3-6; NNUE + Negamax + Alpha-Beta; error decreasing from around 0.402.
Hard	Blunder-Proof, Vanguard-AI, Iron-Rank Engine, Tal-Resonance, Nimzo-Mind, Oracle-Chess, Checkmate-Protocol, Alpha-Nexus	Depth 5-9; NNUE + Negamax + Alpha-Beta; error from around 0.154 down to near zero.
Master	Bronze Cup Elite, Silver Cup Master, Gold Cup Immortal plus cup-specific AI bot names	Cup Round Robin. Slight error only.
Grandmaster	Final Grandmaster boss list to be locked from client-approved names	Zero error. Deterministic highest search profile.

Engine Architecture Details

Exact routing and rule engine design

Beginner Engine: SCE + PST on Pawn/Bishop/Knight

- Beginner must not use NNUE. It must feel simple and beatable.
- SCE means Static Chess Evaluation / simple handcrafted evaluation.
- PST must apply only to Pawn, Bishop and Knight. Rook, Queen and King should use material/basic safety only at Beginner level.
- The bot must still obey every legal chess rule. Weak does not mean illegal.
- Randomness is allowed but must not create repeated left-right loops.

Learner Engine: SCE + All-Piece PST

- Learner still does not use NNUE.
- Learner uses SCE with PST for Pawn, Bishop, Knight, Rook, Queen and King.
- Learner should be stronger than Beginner but still forgiving.
- Learner downlevel rule is 3 consecutive losses on the current level, not 2.

Intermediate to Grandmaster Engine: NNUE + Negamax + Alpha-Beta

- Intermediate, Hard, Master and Grandmaster must route to the neural/NNUE engine.
- Search must use Negamax with Alpha-Beta pruning.
- Error/noise is a difficulty parameter, not a rule-breaking parameter.
- Grandmaster must have zero error and deterministic best move behavior.
- Master can have slight error but should still feel strong.

HalfKP / HalfKA and HCE-NNUE Continuity

- When SCE/HCE and NNUE are mixed across tiers, evaluation discontinuity can occur. A player may feel the bot suddenly changes personality between Learner and Intermediate.
- Use consistent feature mapping, scaling, and score normalization when switching from SCE/PST to NNUE.
- NNUE should keep a stable feature scheme such as HalfKP-style mapping. If HalfKA is used, document it separately and keep tests.
- Do not silently mix unrelated score scales. Convert SCE scores and NNUE centipawn scores to a common scale before difficulty noise is applied.

Rust Rule Engine Conversion from AI_rules

Rule area	Must be implemented in Rust	Acceptance test
Legal moves	All pieces, captures, blockers, own king safety	Every generated move must be legal in shakmaty.
Check	Detect when active king is attacked	Known check positions pass.
Checkmate	No legal move while in check	Mate-in-1 and normal mate fixtures pass.
Stalemate	No legal move while not in check	Academy stalemate must not start already finished unless teaching that exact final state.
Castling	Rights, blocked squares, through-check, king/rook moved	Illegal castling blocked; legal castling works.
En passant	Only immediate legal en passant target	EP tests pass for both colors.
Promotion	Player chooses piece; AI auto chooses queen unless strategy says otherwise	Promotion UI and engine FEN remain valid.
50-move rule	Halfmove clock	Draw detected only when condition is true.
Threefold repetition	Position hash with rights/side/en-passant where relevant	Repetition counted correctly.
Side orientation	If player selects black, logic and board orientation must match	2D and 3D both open from player side.

Progression and Cup System

Exact unlocking, downlevel and tournament behavior

Sequential Unlock Rules

- Only the first Beginner bot is unlocked for a fresh user.
- Winning one bot unlocks only the next bot, not the full tier.
- Completed bots can show Replay/Completed. Current bot can show Current/Challenge. Locked bots must be unclickable.
- Direct routing to a locked bot must be blocked and redirected to the current playable bot.
- Lower completed tiers can be replayed, but future tiers remain locked.

Loss Rules

Level	Loss behavior	Required state field
Beginner	Loss does not send player back. Player retries same level.	none or loss count for stats only
Learner	3 consecutive losses on current level sends player back 1 level.	consecutiveLossesByCharacter or per-tier counter
Intermediate	2 consecutive losses sends player back 1 level.	consecutiveLossesByCharacter
Hard	1 loss sends player back 1 level.	lastResult sufficient but keep audit log
Master	Cup rules apply; failed cup retries same cup unless specified otherwise.	cup points table + results
Grandmaster	Final elite policy; zero error.	grandmaster progress state

Master Cup: Last Method Only - Round Robin / Point Table

- Each cup contains 3 AI bots and 1 real player.
- Everyone plays tournament matches. Player must play each AI bot. AI-vs-AI matches are simulated by Rust /engine/simulate.
- Win = 3 points, Draw = 1 point, Loss = 0 points.
- After all matches, build points table with tie-breakers: points, head-to-head, Sonneborn-Berger/simple strength score, then rating seed if still tied.
- Player rank 1 clears the cup. Player rank 2, 3 or 4 fails and retries the same cup.
- Do not implement Knockout or Ladder. They were options in the source, but the final selected method is Round Robin.

All Past and Current Bugs Included

Client feedback checklist and solution map

Bug / issue	Likely cause	Required solution
Bot repeated same rook left-right	Weak bot selection had no anti-repetition/variety memory.	Root-level anti-loop penalty and recent AI move tracking in Rust.
Beginner took around 30 seconds	Search depth or server waking was too high for low tier.	Beginner max think time $\leq$ 800ms local target; fallback random legal move if timeout.
Rules not updated in real game	Old frontend/client rule logic still active.	Move all legal rules to Rust/shakmaty; TS calls Rust only.
Academy lessons start already solved	Static lesson FENs are final/checkmate/stalemate positions instead of teaching positions.	Create lesson-specific FENs requiring 1-2 moves only; freeze non-required pieces.
Academy lets full game continue	Training mode reused full match state.	Use micro-puzzle engine: expected moves only, then lesson complete.
Customize preview crashes	Preview state likely renders invalid asset/undefined theme.	Type-safe preview schema, error boundary, fallback preview model.
Wrong level sections	Core and Promotion Trial were added but not required.	Remove them. Final tabs only Beginner, Learner, Intermediate, Hard, Master, Grandmaster.
All levels unlocked early	Progression check based on tab/current level incorrectly.	Sequential completion map and direct-navigation guard.
ELO/rating mismatch	Random uneven ratings such as 50/200/150/220.	Use smooth per-tier rating ranges table in this PDF.
Black selected but 3D opens white-side	3D camera orientation not reading player color.	Apply playerColor orientation to 3D camera on match start and next level remount.
Piece flicker on every move	3D/React remounting piece meshes or keys changing unnecessarily.	Stable piece keys and animation diff only changed piece.
AI overview uncertain	Stats panel may be disconnected from actual engine response.	Bind overview to engine_used, depth, error, weights_status, inference_mode.
RTT / waking server confusion	Latency badge shown without explanation.	Rename/tooltip: Server RTT. Green means reachable, not NNUE proof.
Sign-in value not explained	User does not know why to create account.	Small message: Sign in to save progress across devices.
Game stats incomplete	Only total games/wins tracked.	Add white games/wins, black games/wins, draws, losses, checkmates, tier progress.
Price mismatch 199 vs 299	Two deployed links/builds/configs have different plan price.	Single pricing config source and deployment parity check.
Analysis perspective always white	Analysis board ignores player color.	Analysis perspective = playerColor; black player sees black orientation.
Hindi localization incomplete	Some labels like Customize remain English.	Central i18n dictionary; no hardcoded UI strings.
web.app and firebaseapp different builds	Two Firebase Hosting channels/targets not deployed consistently.	Single canonical hosting target or deploy both from same build artifact.
3D board loading stuck	3D asset load/performance blocker.	5-second fallback to 2D; continue game without blocker.
Deployed tensor not proven	Production backend returned placeholder tags.	Render secret file and /engine/move must prove trained/tensor before public release.

Implementation Phases

Exact execution order for the coding agent

Phase	Work
Phase 0 - Freeze and backup	Create branch, backup current build, list current URLs, export Firebase config, mark public release blocked until this plan passes.
Phase 1 - Source-of-truth config	Create Rust/TS shared level config from final tiers. Remove Core/Promotion Trial. Add clean ratings and bot profiles.
Phase 2 - Rust rules engine	Convert AI_rules behavior into Rust with shakmaty. Add all FIDE rule tests. Remove old frontend move validation from gameplay.
Phase 3 - Engine routing	Beginner=SCE/PST partial; Learner=SCE/PST all; Intermediate+=NNUE+Negamax+Alpha-Beta. Add route tests.
Phase 4 - Error and timeout tuning	Set error schedule 0.402 -> 0.154 -> slight -> 0. Add max think times. Prevent Beginner 30-second thinking.
Phase 5 - Progression	Implement sequential unlock and loss/downlevel rules: Learner 3, Intermediate 2, Hard 1. Add state migration.
Phase 6 - Cup Round Robin	Implement 3 AI + 1 real player point-table cups. AI-vs-AI via Rust simulation. Add tie-breakers.
Phase 7 - Academy rewrite	Replace full-game academy states with micro-puzzles requiring 1-2 moves. Fixed pieces. No already-solved check/stalemate unless explicitly final lesson.
Phase 8 - UI bug fixes	CTA, sign-in save message, stats breakdown, pricing single source, Hindi i18n, analysis perspective, RTT tooltip.
Phase 9 - 3D reliability	Fix black orientation, flicker, 3D fallback to 2D, stable piece keys, no infinite loading.
Phase 10 - Hosting parity	Make web.app and firebaseapp deploy same build/config/symbols/pricing. Document canonical URL.
Phase 11 - Deployed tensor proof	Set NNUE_WEIGHTS_PATH and prove weights_status=trained, inference_mode=tensor on deployed backend.
Phase 12 - Full QA	Run all commands, Android test, web test, manual checklist, create final release decision report.

Acceptance Tests and Verification

100 percent clarification checklist

Automated Commands

Frontend:

```
npm run security:scan
npm run build
npx vitest run
npx cap sync android
```

Rust:

```
cd src-rust
cargo fmt -- --check
cargo check
cargo test
```

Must-Pass Manual Tests

- Fresh install: only Beginner level 1 is playable.
- Beginner level 1 uses SCE + PST only on Pawn/Bishop/Knight and responds quickly.
- Learner uses all-piece PST and drops one level only after 3 consecutive losses.
- Intermediate uses NNUE + Negamax + Alpha-Beta and drops after 2 consecutive losses.
- Hard drops after 1 loss and never illegal-moves.
- Master cup runs 3 AI + 1 player Round Robin with correct points table.
- Grandmaster shows zero error behavior and deterministic response for same FEN/settings.
- All real-game moves follow legal Rust rule validation.
- Academy lessons teach one concept with fixed pieces and no already-finished puzzle states.
- Black player sees 2D and 3D board from black side and analysis from black perspective.
- 3D fallback to 2D appears within 5 seconds if loading is slow.
- No flicker on normal piece moves.
- Sign-in prompt explains saved cross-device progress.
- Stats show total, white-side, black-side, wins, losses, draws and checkmates.
- Pricing is same on web.app and firebaseapp.
- Hindi localization has no stray English labels for translated mode.
- RTT badge has tooltip and does not claim NNUE proof.
- Deployed backend /engine/move proves trained/tensor before public release.

Prompt Pages

Copy-paste prompts for the implementation agent

MASTER IMPLEMENTATION PROMPT FOR CLASH OF CROWNS

You are working on Clash of Crowns. Follow this instruction exactly. Do not invent new tiers, do not change architecture, and do not add JavaScript chess engine logic.

FINAL LEVEL ORDER:

1. Beginner
2. Learner
3. Intermediate
4. Hard
5. Master
6. Grandmaster

REMOVE completely:

- Core
- Promotion Trial

LANGUAGE RULES:

- Frontend: TypeScript only.
- Backend/engine: Rust only.
- No JavaScript chess engine logic in production.
- AI_rules.txt is only a reference. Convert its behavior into Rust using a Rust chess library such as shakmaty.
- TypeScript may handle UI, routing, API adapter, state display and localization. It must not be the main chess engine.

ENGINE ROUTING:

- Beginner: SCE/static evaluation with PST only for Pawn, Bishop, Knight. No NNUE.
- Learner: SCE/static evaluation with PST for all pieces. No NNUE.
- Intermediate: NNUE/neural evaluation + Negamax + Alpha-Beta pruning.
- Hard: NNUE/neural evaluation + Negamax + Alpha-Beta pruning.
- Master: NNUE/neural evaluation + Negamax + Alpha-Beta pruning with slight error.
- Grandmaster: NNUE/neural evaluation + Negamax + Alpha-Beta pruning with zero error.

ERROR / DIFFICULTY:

- Intermediate starts around error 0.402 / 0.40 and decreases by bot profile.
- Hard reduces around 0.154 down toward near zero.
- Master has slight error only.
- Grandmaster has exactly zero error.
- Error must never cause illegal moves.

PROGRESSION:

- Beginner: win = next level, loss = stay same level.
- Learner: 3 consecutive losses = down one level, win = next level.
- Intermediate: 2 consecutive losses = down one level, win = next level.
- Hard: 1 loss = down one level, win = next level.
- Master: cup rules apply.
- Grandmaster: final elite tier.

CUP SYSTEM:

Use only Round Robin / Point Table.
Each cup has 3 AI bots + 1 real player.
Win = 3 points, Draw = 1 point, Loss = 0 points.
Player rank 1 clears cup.
Player rank 2/3/4 fails cup and retries same cup.
AI-vs-AI matches must be simulated through Rust /engine/simulate.

RATINGS:

Beginner: 300-700, 5 bots.
Learner: 700-1000, 5 bots.
Intermediate: 1100-1500, 8 bots.
Hard: 1500-2000, 8 bots.
Master: 2000-2500, 3 cups.

Grandmaster: 2500+, final elite bots.
Do not use broken values like 50, 150, 200, 220.

BUGS TO FIX:

- Real game rules still old/incomplete.
- Academy has wrong already-solved states and full-game behavior.
- Customization preview crashes.
- Beginner/weak bots may loop or take too long.
- Black side 3D orientation wrong.
- Piece movement flicker.
- Sign-in save progress message missing.
- Stats missing white/black breakdown.
- Price mismatch 199/299.
- Analysis always white perspective.
- Hindi localization incomplete.
- web.app and firebaseapp mismatch.
- RTT/waking server text unclear.
- 3D board must fallback to 2D.

OUTPUT REQUIRED:

Implement in phases. After each phase run tests. Final report must include changed files, tests, manual verification and release decision.

PROMPT PAGE 2 - RUST RULE ENGINE

Convert AI_rules behavior into Rust. Do not paste JavaScript or HTML into the app.

Implement a Rust rules module using shakmaty or equivalent:

- FEN parse/serialize
- legal move generation
- check/checkmate/stalemate
- castling rights and through-check validation
- en passant
- promotion
- 50-move rule
- repetition validation
- side-to-move handling
- move execution and legal board state validation

Create endpoint-compatible types for frontend:

- MoveRequest
- MoveResponse
- LegalMove
- GameStatus
- RuleViolation

Frontend TypeScript must call Rust and render results. It must not run production chess rules independently.

Tests required:

- legal_pawn_moves
- legal_knight_moves
- legal_bishop_moves
- legal_rook_moves
- legal_queen_moves
- legal_king_moves
- castling_blocked_by_check
- en_passant_valid_only_immediately
- promotion_choice_valid
- checkmate_detected
- stalemate_detected
- fifty_move_draw
- repetition_draw
- illegal_move_rejected
- frontend_cannot_bypass_locked_or_illegal_move

PROMPT PAGE 3 - ENGINE ROUTING AND BOT PROFILES

Create one Rust-side source of truth for bot profiles. Each bot profile must include:

- id
- display name
- tier
- rating
- engine type
- depth
- max_think_time_ms
- error/noise value
- pst mode
- unlock group

PST modes:

- beginner_partial: Pawn, Bishop, Knight only.
- learner_all: Pawn, Bishop, Knight, Rook, Queen, King.
- nnue: NNUE feature evaluation.

Engine types:

- sce_partial_pst
- sce_all_pst
- nnue_negamax_ab

Routing:

Beginner -> sce_partial_pst

Learner -> sce_all_pst

Intermediate/Hard/Master/Grandmaster -> nnue_negamax_ab

Never allow Grandmaster error > 0.

Beginner and Learner must not call NNUE.

Intermediate and above must call NNUE when weights are valid, with fallback documented.

Add tests proving each tier routes to the expected engine.

PROMPT PAGE 4 - PROGRESSION AND LOCK SYSTEM

Implement progression state in TypeScript with a Rust-compatible schema. Do not store fragile UI-only flags as the source of truth.

State must track:

- current tier
- current bot/cup
- completed bots
- consecutive losses per tier/bot
- total games
- wins/losses/draws
- white-side stats
- black-side stats
- cup table state
- guest/local save metadata
- cloud sync metadata if login is active

Rules:

Beginner: loss stay same, win next.

Learner: 3 consecutive losses down one level, win next.

Intermediate: 2 consecutive losses down one level, win next.

Hard: 1 loss down one level, win next.

Master: cup rules.

Grandmaster: final tier.

Locked bots must be locked visually and logically. Direct navigation to a locked bot must redirect to the current playable bot.

Add tests for fresh start, win unlock, loss counters, downlevel, locked click, direct route bypass and local save persistence.

PROMPT PAGE 5 - MASTER CUP ROUND ROBIN

Use only the Point Table / Round Robin method.

Cup format:

- 3 AI bots + 1 real player.
- Player plays all 3 AI bots.
- AI bots also play/simulate each other.
- Win = 3 points.
- Draw = 1 point.
- Loss = 0 points.
- Rank 1 clears cup.
- Rank 2, 3 or 4 means cup failed and player retries same cup.

Implementation:

- Cup state must store all pairings, results, points and tie-breakers.
- /engine/simulate must produce AI-vs-AI results.
- UI must show table clearly.
- Do not use Knockout.
- Do not use Ladder/Gauntlet.

Tests:

- cup_creates_four_participants
- player_has_three_matches
- ai_vs_ai_simulations_complete
- points_table_correct
- player_rank_1_unlocks_next_cup
- player_rank_2_retries_same_cup
- tie_breaker_deterministic

PROMPT PAGE 6 - UI, LOCALIZATION AND DEPLOYMENT FIXES

Fix these UI issues without redesigning the whole game:

- Add small sign-in message: "Sign in to save your progress across devices."
- Add stats breakdown: total games, wins, losses, draws, white wins, black wins, white games, black games, checkmates.
- Use one pricing config so 199/299 mismatch cannot happen.
- Analysis board perspective must match player side.
- Hindi mode must use centralized translations. No mixed hardcoded English labels like Customize.
- RTT badge should show tooltip: "Server round-trip time. Green means server is reachable. It does not prove NNUE tensor mode."
- web.app and firebaseapp must be deployed from same build artifact and config.
- 3D board must respect player side and fallback to 2D within 5 seconds if slow.
- Piece movement flicker must be fixed by stable keys and diff-based animation.

Add tests or manual QA evidence for each item.

PROMPT PAGE 7 - FINAL QA AND RELEASE DECISION

After implementation, run:

Frontend:

- npm run security:scan
- npm run build
- npx vitest run
- npx cap sync android

Rust:

- cargo fmt -- --check
- cargo check
- cargo test

Manual QA:

- Fresh guest start.
- Beginner first bot only unlocked.
- Beginner engine responds quickly and uses only Pawn/Bishop/Knight PST.
- Learner all-piece PST and 3-loss downlevel.
- Intermediate NNUE route and 2-loss downlevel.
- Hard NNUE route and 1-loss downlevel.
- Master Round Robin cup.

- Grandmaster zero-error.
- Real rules valid.
- Academy fixed.
- Black side 2D/3D/analysis correct.
- 3D fallback works.
- No flicker.
- Stats, sign-in message, pricing, Hindi, hosting parity verified.
- Deployed /engine/move returns trained/tensor before public release.

Final release labels:

- If all pass: RELEASE_CANDIDATE.
- If deployed tensor not proven: INTERNAL_TESTING_ONLY.
- If rule/progression bugs remain: NOT_READY_WITH_BLOCKERS.

Final Release Gate

When the game is actually ready

- The game is not public-release ready merely because this PDF exists.
- The game becomes release candidate only when this plan is implemented and all acceptance tests pass.
- The most important proofs are: Rust rules correct, final level routing correct, progression/loss rules correct, Round Robin cup correct, UI blockers fixed, Android works, hosting URLs match, and deployed NNUE returns trained/tensor for Intermediate+.
- If deployed NNUE still returns placeholder, the game can remain internal testing only, not final public production.
- If any old JavaScript rule engine remains active in production gameplay, this plan is not completed.

Final owner requirement statement: After implementation, Clash of Crowns should be playable according to the connected requirements: Beginner and Learner use SCE/PST learning engines; Intermediate onward uses NNUE + Negamax + Alpha-Beta; progression follows exact loss rules; Master uses Round Robin cups; UI/academy/analysis/orientation/stats/deployment issues are fixed; frontend is TypeScript only and engine is Rust only.